Lab Assignment 2 – Async Weather Tracker

Course: Web Dev II (Advanced JS & React)
Unit Covered: Unit–2
Marks: 2.5
Mode: 100% In-Class (Lab Practicals)
Time: 30–40 minutes

Assignment Overview

The **Async Weather Tracker** is a hands-on lab assignment designed to help students understand **asynchronous programming in JavaScript** and how JavaScript behaves at runtime.

In this lab, students will build a weather information system using **Vanilla JavaScript** and the **Fetch API**, focusing on **async/await, promises, error handling, event loop behavior, and Local Storage**—without using any frameworks or external libraries.

This assignment simulates real-world API interaction and helps students debug and analyze execution order using browser developer tools.

Learning Objectives

- Understand asynchronous JavaScript execution flow
- Use `fetch()` with `async/await` to call public APIs
- Handle promise states using `.then()`, `.catch()`, and `try...catch`
- Observe JavaScript runtime behavior using console logs
- Analyze the **event loop, call stack, and task queue**
- Store and retrieve data using **Local Storage**
- Handle API errors and invalid user input gracefully

🛠️ Technology Stack

- HTML5 – Structure of the web page
- CSS3 – Basic styling (no frameworks allowed)
- Vanilla JavaScript – DOM manipulation and event handling

⚠️ Restrictions:

- ❌ No UI libraries (Bootstrap, Tailwind, etc.)
- ✅ Only plain HTML, CSS, and JavaScript

📄 Functional Requirements

1️⃣ Weather Search Interface

- Input field to enter a city name
- Search button to fetch weather details
- Display city name, temperature, and weather condition

2️⃣ Asynchronous API Handling

- Fetch weather data from a public weather API using:
  - `async/await`

3️⃣ Error Handling & Promise States

- Handle invalid city names gracefully
- Display a user-friendly error message for:
  - Network errors
  - Invalid API responses
- Demonstrate error handling using:
  - `.then()` / `.catch()`
  - `try...catch`

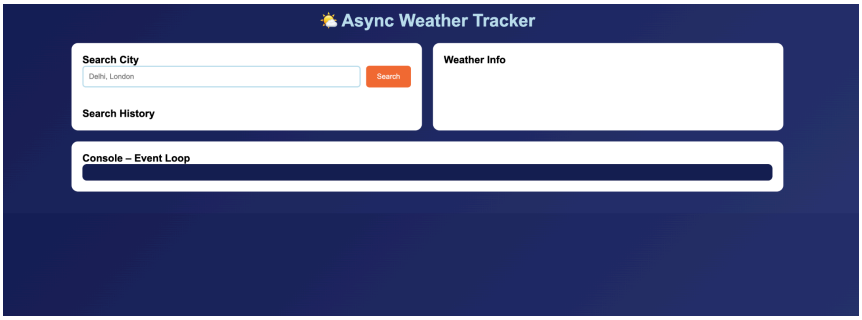
4️⃣ Local Storage – Search History

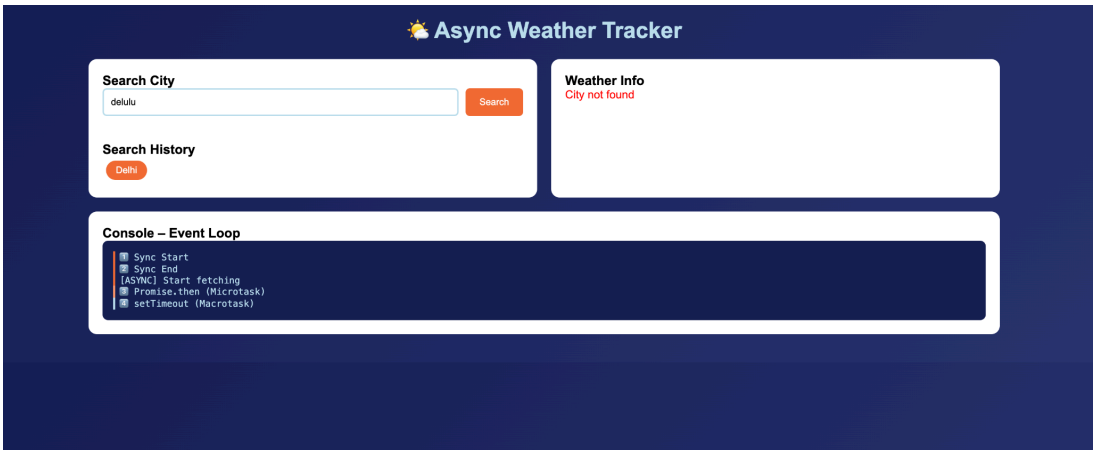
- Store previously searched city names in **Local Storage**
- Display recent searches on page load
- Allow users to click a previous city to re-fetch weather data

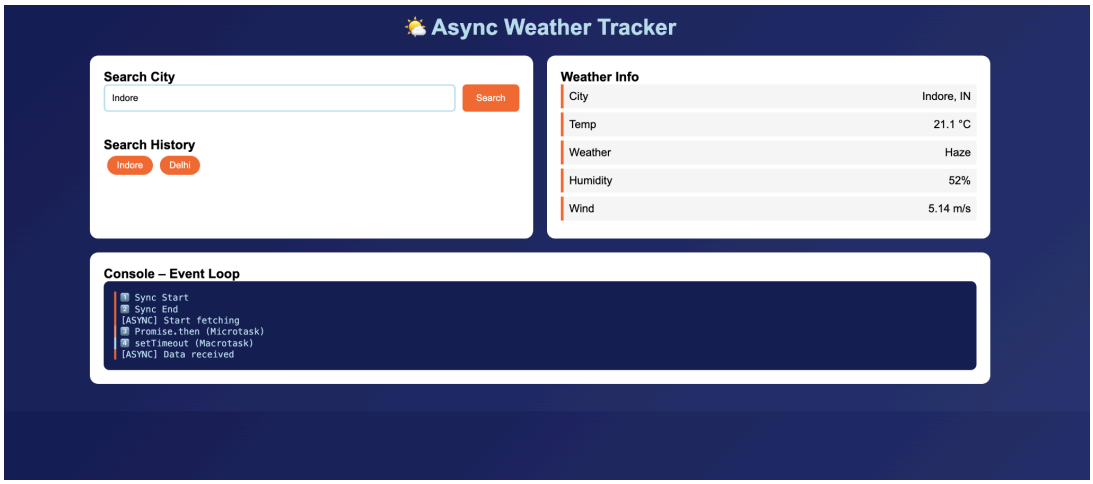
---

5️⃣ Event Loop & Execution Order Analysis

- Use `console.log()` statements to:
  - Track function execution order
  - Observe async vs synchronous behavior
- Include logs before and after the `fetch()` call
- Analyze how the **call stack** and **event loop** work







Video Sample : [Click here to see the Video Demo]

Rubric for Assignment–2 :

Criteria	Marks
Functionality	1.5
UI	0.5
Clean Code	0.5
Total	**2.5**

Happy Coding! 🎯